

School of Engineering, Computing and Mathematics
Oxford Brookes University
COMPT5047: Applied Software Engineering
(Semester 1, 2024-2025)

Resit Coursework Specification

Software Engineering of a Modern Computer Application

1 Assessed Learning Outcomes

This resit coursework counts as **100%** of the total assessment of this module. It is designed to assess your attainment of the following learning outcomes.

1	Demonstrate an understanding of the role of requirements analysis and specification in software engineering and to be able to use this knowledge to create use case models and functional models of computer applications.
2	Demonstrate an understanding of the relationship between requirements and design and to be able to apply the knowledge to create structural and behavioural models of computer applications.
3	Critically evaluate and utilise design paradigms of object-oriented analysis and design, component-based design, and service-oriented design.
4	Use software modelling language such as UML and modelling tools in the context of model-driven software engineering.
5	Work in a group to apply the knowledge and skills developed in this module

Engineering Council AHEP4 LOs assessed

C3	Select and apply appropriate computational and analytical techniques to model complex problems, recognising the limitations of the techniques employed
C5	Design solutions for complex problems that meet a combination of societal, user, business and customer needs as appropriate. This will involve consideration of applicable health & safety, diversity, inclusion, cultural, societal, environmental and commercial matters, codes of practice and industry standards
C6	Apply an integrated or systems approach to the solution of complex problems
C14	Discuss the role of quality management systems and continuous improvement in the context of complex problems
C16	Function effectively as an individual, and as a member or leader of a team

Roham Mostakhdeemin

Student ID: 19275124

Customer SubSystem

1. Project Management and SE tools

1.1. Name of UML modelling tool used in each task:

- Task 3: Eclipse UML designer has been used to model both use case diagram and activity diagram
- Task 4: Eclipse UML designer was again used to make component diagram and interfaces as I had some experience from working on the same coursework last semester.
- Task 5: Due to limitations in Eclipse UML designer capabilities, for both the class diagram and sequence diagram, Eclipse Papyrus has been utilised to model the application.

1.2. URL of Github Repository:

the repository can be found at:

<https://github.com/RohamDemin/COMP5047> I have also sent an invitation to Professor Hong Zhu.

A note about commits to this repository:

Since the nature of files (screenshots, MS Word documents, Eclipse and UML designer project files) being uploaded is not editable via Git clients in place, they have been replaced with newer versions for every commit. The commit history is visible and will provide history of my work.

1.3. List of Folders and Files on Github repository

- **CloudTables-Customer** Folder: this folder houses my original coursework and can be ignored
- **task 2** Folder houses my written section files for quality requirements. The file named quality Requirements final.docx is the finalised version found in this report.

[COMP5047 / task 2](#) / 

RohamDemin finalised Version of quality requirements added	
Name	Last commit message
..	
 Quality Requirements Final.docx	finalised Version of quality requirements added
 Quality Ver 2.docx	Task 2 Updated.

- **Task 3** folder houses two screenshots of the use case diagram and activity diagram for task 3, named Resit Activity diagram.jpg and use case diagram.jpg both files are taken from UML designer software

COMP5047 / Task 3 / 

RohamDemin Activity Diagram Edited

Name

 .. Resit Activity Diagram.jpg Use Case Diagram.jpg

- **Task 4** folder includes final version of architectural diagram and textual description of components and interfaces. The files named Architecture.jpg and Final textual description.docx are the final versions.

COMP5047 / Task 4 / 

RohamDemin Final Versions of diagram and textual description uploaded

Name

 .. Architecture.jpg Component interface list.docx Final textual description.docx

- Task 5 folder has a subfolder named CloudTables-Customer which is the project folder for Eclipse Papyrus. Task 5 Folder also has three screenshots for the class diagram and sequence diagrams.

[COMP5047](#) / Task 5 /

 **RohamDemin** screenshots of class and sequ

Name
 ..
 CloudTables-Customer
 CloudTables-Customer class diagram.jpg
 CloudTables-Customer sequence 1.jpg
 CloudTables-Customer sequence 2.jpg

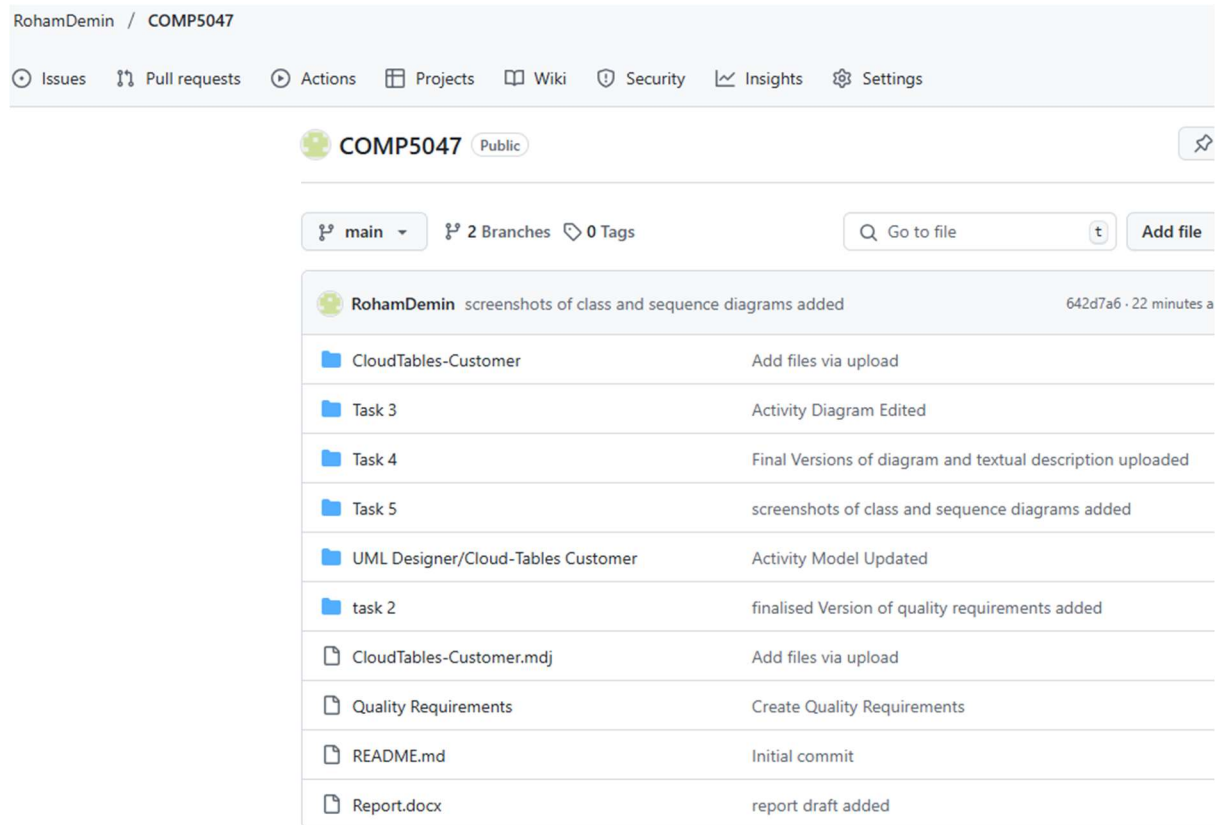
- **UML Designer and its subfolder Cloud-Tables Customer** Folder houses the project Files for UML designer software that is used for task 3(use case and activity diagram) and task 4(architectural design diagram)

[COMP5047](#) / UML Designer /

 **RohamDemin** Activity Model Updated

Name
 ..
 Cloud-Tables Customer

- **CloudTables-Customer.mdj**: file is for StarUML software which I experimented with and is not used for final report and thus can be ignored.
- **Quality Requirements** text file is an earlier version of quality requirements for task 2 and can be ignored.
- **README.md**: initial git commit message
- **Report.docx**: is a copy of this report in docx format



Above is a picture of the repository in its entirety.

2. Specification of Quality Requirements

Below, quality requirements are categorised according to functional requirements described in the case study.

2.1 FR-RC-1: Search for Restaurants

a. Security and Privacy Protection

- Location Data Privacy: The user's current location must be accessed only with permission and discarded immediately after search completion.
- Encrypted Queries: All search queries must be transmitted over HTTPS using TLS 1.3 to protect user preferences (e.g., cuisine or location).
- No User Tracking: Anonymous users' searches must not be logged or tracked beyond session duration. (Confirmed through log audit and privacy policy review.)
- Public restaurant data such as name, address, cuisine type, table availability, and location should be read-only to search operations and customers are not allowed to create, update, or delete any restaurant-related information.

b. Performance

- Search Response Time: For 95% of searches using any filter (name, location, cuisine, price, availability), results must be returned within 2 seconds. (Measured using automated response time monitoring.)
- Geo-based Filter Efficiency: Location-based filtering (current location and distance) must process results under 2.5 seconds for distances up to 50 km. (Tested with mocked GPS locations.)
- Concurrent Searches: The search service must handle at least 50 concurrent user queries without degraded response time. (Load tested with concurrent user simulations.)

c. Reliability

- Result Accuracy: Search results must return 100% matching records for each specified filter (e.g., only show restaurants that have tables for 4 people under £30). (Unit tests and automated filter validation.)
- Consistency Under Load: Identical queries must return consistent results with a maximum 1% variation under load. (Tested by repeating filtered queries in parallel.)
- Failure rate should be kept under 0.1% for total search queries.
- Fail Resilience: In case of service failure, the search feature should fall back to a cached data store within 5 seconds. (Tested with fault injection in microservice architecture.)
- Availability target for search functionality should be at least 99.5% uptime. This can be tested with automated testing tools.

d. Scalability

- Regional Load Support: The search system must support 10,000+ restaurants within a single region (e.g., London) without exceeding 3 seconds per query. (Stress tested with large dataset loads.)
- Peak Time Readiness: The search service must scale up to 5x average weekday traffic during peak times (e.g., Friday evening) while maintaining SLA of 2 seconds response time. (Simulated peak-time testing in staging.)
- Global Query Handling: Must support customer queries from at least 10 different time zones simultaneously with no regional performance degradation. (Geographic testing using distributed performance agents.)

2.2 FR-RC-2: Booking a Table

a. Security and Privacy Protection

- Authenticated Booking Only: Only registered/authenticated users can book a table. (Verified via access control tests.)
- customers should only be able to view, modify, cancel their past booking history and no other users' booking data.
- Secure Booking Data: Booking data (user ID, table ID, date/time) must be encrypted at rest with AES-256. (Confirmed through database configuration and security audit.)
- Access Control on Layout: Room layout and availability data must not expose other customers' bookings. (Penetration tested for data isolation.)

b. Performance

- Booking Response Time: 95% of booking operations must complete within 3 seconds, including table availability check and reservation confirmation. (Performance test via API benchmarks.)

- Room Layout Loading: Room layout and table availability must render within 2.5 seconds. (Measured via UI load testing on low-bandwidth connections.)
- UI Feedback Loop: Confirmation status (booked, not available) must be displayed in real-time (max 5 seconds delay). (Live test with event propagation.)

c. Reliability

- No Double-Booking: The system must prevent two users from booking the same table at the same time. (Concurrency testing under race conditions.)
- Persistent Reservation: Once a table is booked, the reservation must persist for 10 minutes even in case of app closure or network loss. (Tested through offline simulation and retry logic validation.)
- Availability Consistency: Table availability must be updated globally within 5 seconds across all customer devices. (Monitored via real-time sync validation tools.)
- The failure rate for booking operations should not exceed 0.05%, and the MTTR for booking-related service failures must be less than 1 minute.
- The booking function is mission-critical and must be available 99.9% of the time

d. Scalability

- High Booking Volume: System must support up to 200,000 table bookings/day across all restaurants with sub-second database latency. (Load testing at capacity.)
- Table Allocation for Large Venues: Booking must function properly for restaurants with up to 300 seats and multiple floors. (Tested with varied layout schemas.)
- Peak Reservation Traffic: Must handle 10x average load during holiday/event periods (e.g., Valentine's Day) with no performance degradation. (Scalability stress testing with simulated spikes.)

2.3 FR-RC-3: Pre-Order of Food

a. Security and Privacy Protection

- Pre-Order Access Control: Only users with an active booking can access the pre-order feature. (Functionally tested via backend policy enforcement.)
- Each customer may **create**, **edit**, and **cancel** their own pre-order up to 60 minutes before their scheduled arrival.
- Sensitive Info Encryption: Dietary restrictions and allergy info must be encrypted both in transit and at rest. (Inspected using encryption validation tools.)
- No other customers or restaurants have access to a user's private order details, which may include selected dishes, dietary notes, and quantities.
- Revision Audit Trail: Any modification to the pre-order must be logged securely for traceability. (Tested through audit log review.)

b. Performance

- Pre-Order Confirmation: 99% of submissions must complete within 2.5 seconds. (Measured by API response time testing.)
- Real-Time Menu Updates: Updated dish availability or "Today's Special" must sync across all user apps within 2 seconds. (Verified through real-time pub/sub model simulation.)
- Once the customer confirms their selections, the pre-order must be submitted and acknowledged within 2 seconds.

- Customisation Load: Up to 5 simultaneous customisations per dish must process without lag. (Tested with JSON payload simulations.)

c. Reliability

- Offline Sync Support: Pre-orders composed offline must auto-sync within 10 seconds of reconnecting. (Tested via simulated airplane mode behaviour.)
- Order Consistency: The submitted pre-order must be retrievable in identical form at the restaurant terminal. (Tested via backend state comparison.)
- Revision Reliability: Pre-orders must support edits up to 60 minutes before arrival with no more than 1% error rate.
- The pre-order system must maintain 99.8% uptime, and the failure rate for order submissions should not exceed 0.1%.
- Any service disruption should be resolved with a mean time to recovery under 2 minutes.

d. Scalability

- Order Throughput: Must handle 1000 pre-orders per hour per restaurant across the system to cater for peak-time loads. (Load tested at regional cluster level.)
- Complex Menus: Restaurants with 100+ menu items and 10+ categories must be fully supported without delay. (Stress tested with dynamic JSON structures.)
- Large Pre-Order Capacity: Users can add up to 20 items in a single order without performance impact. (Measured with user session logs.)

2.4 FR-RC-4: Customer Feedback

a. Security and Privacy Protection

- Verified Users Only: Feedback must be accepted only from users who completed a booking and order. (Role-based access enforcement test.)
- Each customer can create, edit, or delete only their own reviews, which may include text comments, star ratings, and optional images.
- Image Upload Validation: All images must be scanned for viruses and malicious content (including offensive language/images) before being stored on database and before public display.
- Submitted feedback is publicly visible, but customer identities are masked using pseudonyms.

b. Performance

- Submission Time: 95% of feedback submissions (text + images) must complete in under 2 seconds. (Measured via client-side timers.)
- Maximum image size is limited to 5MB, and each feedback can be attached with 10 images.
- Media Compression: Uploaded images must compress and preview in-app within 3 seconds. (Tested with 1–5MB files across devices.)
- Feed Load: Public feedback for a restaurant must load in under 2 seconds for 50 entries. (UI test with pagination and lazy loading.)

c. Reliability

- Feedback Persistence: All feedback must be stored with a >99.9% durability guarantee. (Confirmed through database replication tests.)
- Edit Availability: Feedback must be editable within 15 minutes of submission. (Functional tested with time-based access control.)
- Image Integrity: Uploaded images must render correctly across devices with zero distortion. (Tested across multiple screen resolutions.)

- The feedback service must be available at least 99.5% of the time. The failure rate for submissions should be less than 0.1%, and the mean time to recovery in the event of a failure should not exceed 2 minutes.

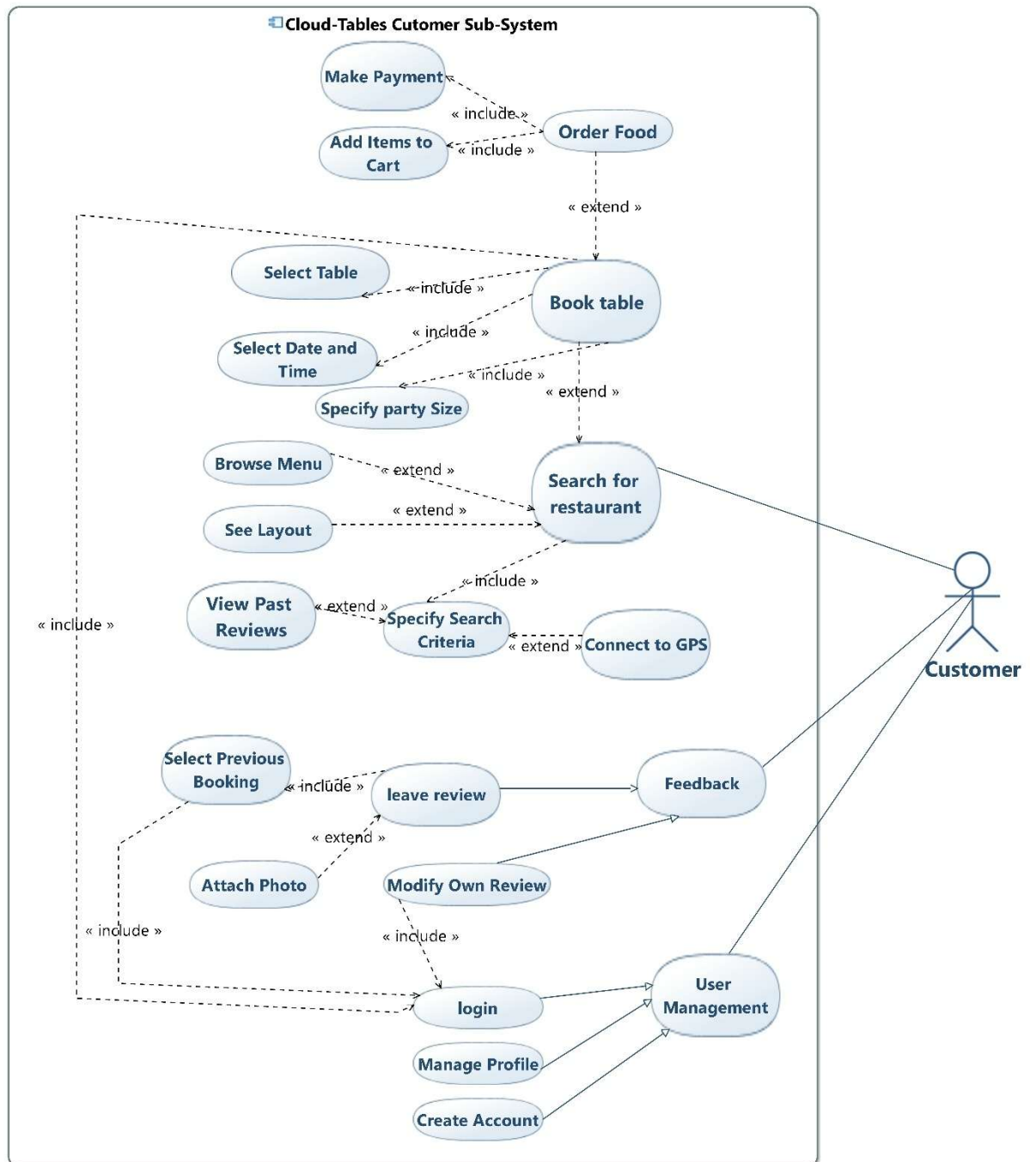
d. Scalability

- Mass Feedback Support: System must scale to 1 million+ feedback entries with <5% latency impact on query performance. (Stress tested with indexed search.)
- Social Integration: Must support share-to-social (e.g. Facebook, X, Instagram) for feedback posts. (API integration tested with OAuth logins.)
- Global Access: Feedback must be globally accessible with <300ms latency in all supported regions. (Monitored with global CDN endpoints.)

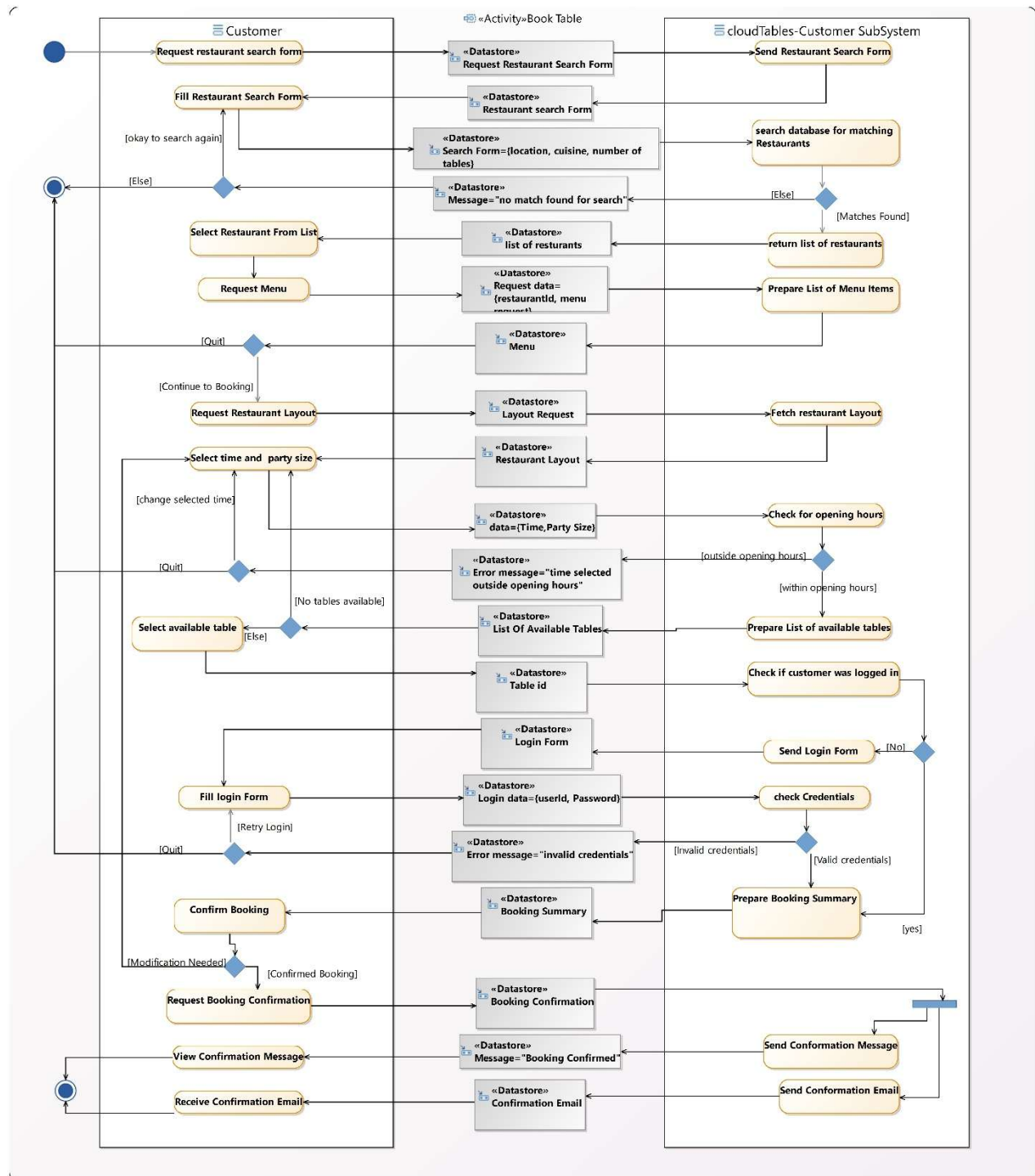
3. Functional Model

3.1. Use Case Diagram

The use case diagram was modelled using UML designer and a screenshot can be found below:

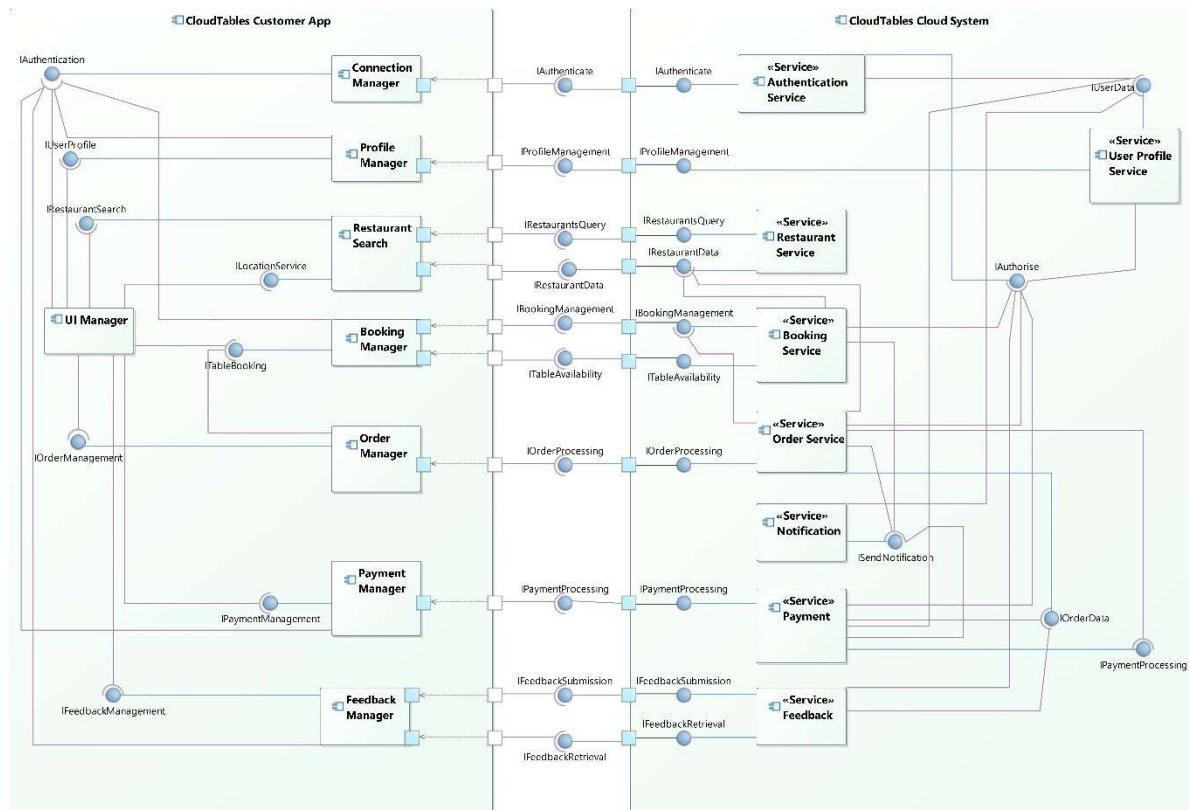


3.2. Activity Diagram



4. Architectural Design

4.1. UML Component Diagram



4.2. Textual Specification of components

Component	UI Manager
Description	Manages the user interface elements and interactions for the mobile app, enabling navigation and displaying information to users. It serves as the presentation layer that coordinates all visual elements and user inputs.
Stereotype	Component
Required Interfaces	<i>IAuthentication</i> , <i>IUserProfile</i> , <i>IRestaurantSearch</i> , <i>ITableBooking</i> , <i>IOrderManagement</i> , <i>IFeedbackManagement</i> , <i>IPaymentManagement</i> , <i>ILocationSearch</i>
Provided Interfaces	None

Component	Connection Manager
Description	Handles authentication and identity management on the client side, providing secure access to the system's functions. It manages network connectivity and secure communication with cloud services.
Stereotype	Component
Required Interfaces	<i>IAuthenticate</i>
Provided Interfaces	<i>IAuthentication</i>

Component	Profile Manager
-----------	-----------------

Description	Manages user profile data on the client side, including viewing and updating personal information. It synchronizes local profile data with the cloud service and handles profile-related operations.
Stereotype	Component
Required Interfaces	IProfileManagement, IAuthentication
Provided Interfaces	IUserProfile

Component	Restaurant Search
Description	Provides restaurant discovery functionality with location-based features, allowing users to find suitable dining options. It implements search filters and manages the display of restaurant information including menus.
Stereotype	Component
Required Interfaces	IRestaurantQuery, IRestaurantData
Provided Interfaces	IRestaurantSearch, ILocationService

Component	Booking Manager
Description	Handles table booking operations on the client side, enabling users to reserve and manage restaurant tables. It coordinates the booking process and maintains booking state on the device.
Stereotype	Component
Required Interfaces	IBookingManagement, ITableAvailability, IAuthentication
Provided Interfaces	ITableBooking

Component	Order Manager
Description	Manages food ordering functionality on the client side, allowing users to browse menus and place orders. It handles order creation, modification, and status tracking.
Stereotype	Component
Required Interfaces	IOrderProcessing, ITableBooking
Provided Interfaces	IOrderManagement

Component	Feedback Manager
Description	Implements feedback submission features on the client side, enabling users to rate restaurants and share experiences. It manages review creation, photo uploads, and feedback history.
Stereotype	Component
Required Interfaces	IFeedbackSubmission, IFeedbackRetrieval, IAuthentication
Provided Interfaces	IFeedbackManagement

Component	Payment Manager
Description	Handles payment operations on the client side, allowing users to pay for orders securely. It manages payment methods, transaction history, and payment confirmation.

Stereotype	Component
Required Interfaces	IPaymentProcessing, IAuthentication
Provided Interfaces	IPaymentManagement

Component	Authentication Service
Description	Validates user credentials and manages permissions on the cloud side, providing secure identity verification. It handles user authentication, session management, and access control.
Stereotype	Microservice
Required Interfaces	IUserData
Provided Interfaces	IAuthenticate, IAuthorize

Component	User Profile Service
Description	Stores and manages user profile information in the cloud, handling registration and profile updates. It maintains user preferences and provides profile data to other services.
Stereotype	Microservice
Required Interfaces	IAuthorize
Provided Interfaces	IProfileManagement, IUserData

Component	Restaurant Service
Description	Provides restaurant information and search capabilities, maintaining the database of restaurants and menus. It handles restaurant discovery, menu management, and table layout information.
Stereotype	Microservice
Required Interfaces	None
Provided Interfaces	IRestaurantQuery, IRestaurantData

Component	Booking Service
Description	Manages table inventory and availability, processing booking requests and handling real-time table status updates. It coordinates reservations, maintains booking records, and enforces booking policies.
Stereotype	Microservice
Required Interfaces	ISendNotification, IRestaurantData, IAuthorise
Provided Interfaces	IBookingManagement, ITableAvailability

Component	Order Service
Description	Processes food orders and maintains order information, coordinating with booking and restaurant services. It handles order creation, modification, pricing, and status tracking.
Stereotype	Microservice
Required Interfaces	IBookingManagement, IRestaurantData, ISendNotification, IAuthorise

Provided Interfaces	IOrderProcessing, IOrderData
---------------------	------------------------------

Component	Notification Service
Description	Sends notifications to users about bookings, orders, and promotions, managing notification delivery across channels. It handles message formatting, delivery scheduling, and notification preferences.
Stereotype	Microservice
Required Interfaces	IUserData
Provided Interfaces	ISendNotification

Component	Feedback Service
Description	Processes and stores customer feedback and ratings, calculating aggregate ratings for restaurants. It manages review moderation, photo storage, and rating analytics.
Stereotype	Microservice
Required Interfaces	IOrderData
Provided Interfaces	IFeedbackSubmission, IFeedbackRetrieval

Component	Payment Service
Description	Handles payment processing for bookings and orders, providing secure financial transactions. It manages payment methods, transaction records, and integration with payment gateways.
Stereotype	Microservice
Required Interfaces	IUserData, IOrderData, ISendNotification, IAuthorise
Provided Interfaces	IPaymentProcessing

4.3. Textual Specification of Interfaces

Name	IAuthenticate	
Provider	Authentication Service	
Operation	Signature	verifyCredentials(credentials: Credentials): AuthToken
	Function	Check if the user's credentials match the stored credentials. If yes, authentication is successful and returns an AuthToken; otherwise, authentication fails and returns an error message. The AuthToken is a string that contains encrypted user information that will be used to validate subsequent requests.
Operation	Signature	validateToken(token: AuthToken): Boolean
	Function	Verify if the provided authentication token is valid and not expired. If valid, returns true; otherwise, returns false. This operation is used to validate user sessions without requiring re-authentication.

Name	IAuthorize
Provider	Authentication Service

Operation	Signature	checkPermission(token: AuthToken): Boolean
	Function	Check if the user identified by the token has permission to access the specified resource. Returns true if access is permitted; otherwise, returns false. This is used for access control throughout the system.

Name	IProfileManagement	
Provider	User Profile Service	
Operation	Signature	getProfile(userId: String): UserProfile
	Function	Retrieve the profile information for the specified user. Returns a UserProfile object containing personal information, preferences, and account details. This operation is used when displaying user information.
Operation	Signature	updateProfile(userId: String, data: UserProfile): Boolean
	Function	Update the profile information for the specified user with the provided data. Returns true if the update is successful; otherwise, returns false. This operation is used when users modify their profile information.

Name	IUserData	
Provider	User Profile Service	
Operation	Signature	getUserInfo(userId: String): UserData
	Function	Retrieve basic user information for the specified user. Returns a UserData object containing essential user details. This operation is used by various services that need user information.
Operation	Signature	getPreferences(userId: String): UserPreferences
	Function	Retrieve the preferences for the specified user. Returns a UserPreferences object containing user settings and preferences. This operation is used to personalize the user experience.

Name	IRestaurantQuery	
Provider	Restaurant Service	
Operation	Signature	findRestaurants(query: QueryParameters): List<Restaurant>
	Function	Search for restaurants matching the specified query parameters. Returns a list of Restaurant objects that match the criteria. This operation supports filtering by location, cuisine, price range, etc.
Operation	Signature	getNearbyRestaurants(location: Location, radius: Double): List<Restaurant>
	Function	Find restaurants within the specified radius of the given location. Returns a list of Restaurant objects sorted by distance. This operation is used for location-based restaurant discovery.

Name	IRestaurantData	
Provider	Restaurant Service	
Operation	Signature	getRestaurantInfo(id: String): RestaurantDetails
	Function	Retrieve detailed information about the specified restaurant. Returns a RestaurantDetails object containing comprehensive restaurant information including contact details, cuisine type, and business hours.
Operation	Signature	getLayout(restaurantId: String): JSON
	Function	Retrieve the table arrangement for the specified restaurant. Returns a TableLayout object representing the physical layout of tables. This operation is used for table selection during booking.
Operation	Signature	getMenu(restaurantId: String): MenuItem[]
	Function	Retrieve the booking policy for the specified restaurant. Returns a BookingPolicy object containing rules for reservations including deposit requirements, cancellation policy, and maximum party size.
Operation	Signature	getAvailability(restaurantId:String, partySize:Integer, dateTime:DateTime) : List<Table>
	Function	retrieves a list of tables available at the time chosen and number of guests.
Provider	Booking Service	
Operation	Signature	createBooking(request: BookingRequest): BookingConfirmation
	Function	Create a new booking with the details specified in the request. Returns a BookingConfirmation object if successful; otherwise, returns an error message. This operation validates availability, applies booking policies, and reserves the table.
Operation	Signature	cancelBooking(bookingId: String): Boolean
	Function	Cancel the specified booking. Returns true if cancellation is successful; otherwise, returns false. This operation applies cancellation policies and updates table availability.
Operation	Signature	getBookingDetails(bookingId: String): Booking
	Function	Retrieve detailed information about the specified booking. Returns a Booking object containing all booking details. This operation is used when users view their booking information.
Operation	Signature	getUserBookings(userId: String): List<Booking>
	Function	Retrieve all bookings for the specified user. Returns a list of Booking objects associated with the user. This operation is used to display booking history and upcoming reservations.

Name	ITableAvailability
Provider	Booking Service

Operation	Signature	checkAvailability(restaurantId: String, dateTime: DateTime, partySize: Int): List<Table>
	Function	Check which tables are available at the specified restaurant for the given date, time, and party size. Returns a list of available Table objects. This operation is used during the booking process to show available options.

Name	IOrderProcessing	
Provider	Order Service	
Operation	Signature	processOrder(order: OrderRequest): OrderConfirmation
	Function	Process a new food order with the details specified in the request. Returns an OrderConfirmation object if successful; otherwise, returns an error message. This operation validates the order, applies pricing, and submits it to the restaurant.
Operation	Signature	updateOrderStatus(orderId: String, status: OrderStatus): Boolean
	Function	Update the status of the specified order. Returns true if the update is successful; otherwise, returns false. This operation is used to track the progress of orders from creation to completion.

Name	IOrderData	
Provider	Order Service	
Operation	Signature	getOrderDetails(orderId: String): OrderDetails
	Function	Retrieve detailed information about the specified order. Returns an OrderDetails object containing comprehensive order information including items, prices, and status. This operation is used when displaying order information.
Operation	Signature	getOrderAmount(orderId: String): Money
	Function	Calculate the total amount for the specified order. Returns a Money object representing the total cost including taxes and fees. This operation is used during payment processing.

Name	ISendNotification	
Provider	Notification Service	
Operation	Signature	sendSuccessNotification(userId: String, message: String): void
	Function	Send a notification with the specified message about an operation being successful
Operation	Signature	sendFailureNotification(userId: String, message: String): void
	Function	Send a notification with the specified message about an operation being failed.

Name	IFeedbackSubmission	
Provider	Feedback Service	
Operation	Signature	submitFeedback(feedback: FeedbackData): String
	Function	Submit a new feedback entry with the details specified in the feedback data. Returns a feedback ID if submission is successful; otherwise, returns an error message. This operation validates the feedback and associates it with the relevant restaurant and user.
Operation	Signature	uploadPhoto(feedbackId: String, photo: Photo): String
	Function	Upload a photo to be associated with the specified feedback. Returns a photo ID if the upload is successful; otherwise, returns an error message. This operation processes and stores the image with the feedback.

Name	IFeedbackRetrieval	
Provider	Feedback Service	
Operation	Signature	getRestaurantRatings(restaurantId: String): RatingSummary
	Function	Retrieve aggregated ratings for the specified restaurant. Returns a RatingSummary object containing average scores across different categories. This operation is used when displaying restaurant ratings.
Operation	Signature	getUserFeedback(userId: String): List<Feedback>
	Function	Retrieve all feedback submitted by the specified user. Returns a list of Feedback objects. This operation is used to display a user's review history.

Name	IPaymentProcessing	
Provider	Payment Service	
Operation	Signature	processPayment(payment: PaymentRequest): PaymentResult
	Function	Process a payment transaction with the details specified in the request. Returns a PaymentResult object indicating success or failure and transaction details. This operation handles payment authorization and processing with payment gateways.
Operation	Signature	refundPayment(paymentId: String): RefundResult
	Function	Process a refund for the specified payment. Returns a RefundResult object indicating success or failure and refund details. This operation is used for cancelled bookings or order issues.

Name	IAuthentication	
Provider	Connection Manager	
Operation	Signature	login(username: String, password: String): AuthToken

	Function	Authenticate the user with the provided credentials. Returns an AuthToken if authentication is successful; otherwise, returns an error message. This client-side operation delegates to the Authentication Service.
Operation	Signature	logout(): Boolean
	Function	End the current user session. Returns true if logout is successful; otherwise, returns false. This operation invalidates the current authentication token.
Operation	Signature	isAuthenticated(): Boolean
	Function	Check if the user is currently authenticated. Returns true if a valid session exists; otherwise, returns false. This operation is used to determine if the user needs to log in.

Name	IUIInterface	
Provider	UI Manager	
Operation	Signature	displayScreen(screenId: String, data: Any): Void
	Function	Navigate to the specified screen and display it with the provided data. This operation controls the app's navigation and content display.
Operation	Signature	showMessage(message: String, type: MessageType): Void
	Function	Display a message to the user with the specified type (info, warning, error). This operation is used for user notifications and feedback within the app.

Name	IUserProfile	
Provider	Profile Manager	
Operation	Signature	getUserInfo(): UserProfile
	Function	Retrieve the current user's profile information. Returns a UserProfile object containing personal information and preferences. This client-side operation may use cached data or fetch from the server.
Operation	Signature	updateUserInfo(profile: UserProfile): Boolean
	Function	Update the current user's profile with the provided information. Returns true if the update is successful; otherwise, returns false. This operation synchronizes changes with the server.

Name	IRestaurantSearch	
Provider	Restaurant Search	
Operation	Signature	searchRestaurants(criteria: SearchCriteria): List<Restaurant>
	Function	Search for restaurants based on the specified criteria. Returns a list of Restaurant objects that match the search parameters. This client-side operation formats and sends search requests to the server.
Operation	Signature	getRestaurantDetails(id: String): Restaurant

	Function	Retrieve detailed information about the specified restaurant. Returns a Restaurant object containing comprehensive restaurant information. This operation fetches and formats restaurant data from the server.
--	----------	--

Name	ILocationService	
Provider	Restaurant Search	
Operation	Signature	getCurrentLocation(): Location
	Function	Determine the user's current geographic location. Returns a Location object containing coordinates. This operation interfaces with the device's location services.
Operation	Signature	calculateDistance(from: Location, to: Location): Double
	Function	Calculate the distance between two locations. Returns the distance in kilometers. This operation is used for proximity-based restaurant filtering.

Name	ITableBooking	
Provider	Booking Manager	
Operation	Signature	checkAvailability(restaurantId: String, dateTime: DateTime, partySize: Int): List<Table>
	Function	Check which tables are available at the specified restaurant for the given date, time, and party size. Returns a list of available Table objects. This client-side operation formats and sends availability requests to the server.
Operation	Signature	bookTable(restaurantId: String, tableId: String, dateTime: DateTime): Booking
	Function	Book the specified table at the given restaurant for the specified date and time. Returns a Booking object if successful; otherwise, returns an error message. This operation creates a booking request and processes the server response.
Operation	Signature	cancelBooking(bookingId: String): Boolean
	Function	Cancel the specified booking. Returns true if cancellation is successful; otherwise, returns false. This client-side operation sends a cancellation request to the server.

Name	IOrderManagement	
Provider	Order Manager	
Operation	Signature	createOrder(bookingId: String, items: List<OrderItem>): Order
	Function	Create a new order with the specified items for the given restaurant. Returns an Order object if successful; otherwise, returns an error message. This client-side operation formats and sends order requests to the server.
Operation	Signature	updateOrder(orderId: String, items: List<OrderItem>): Order

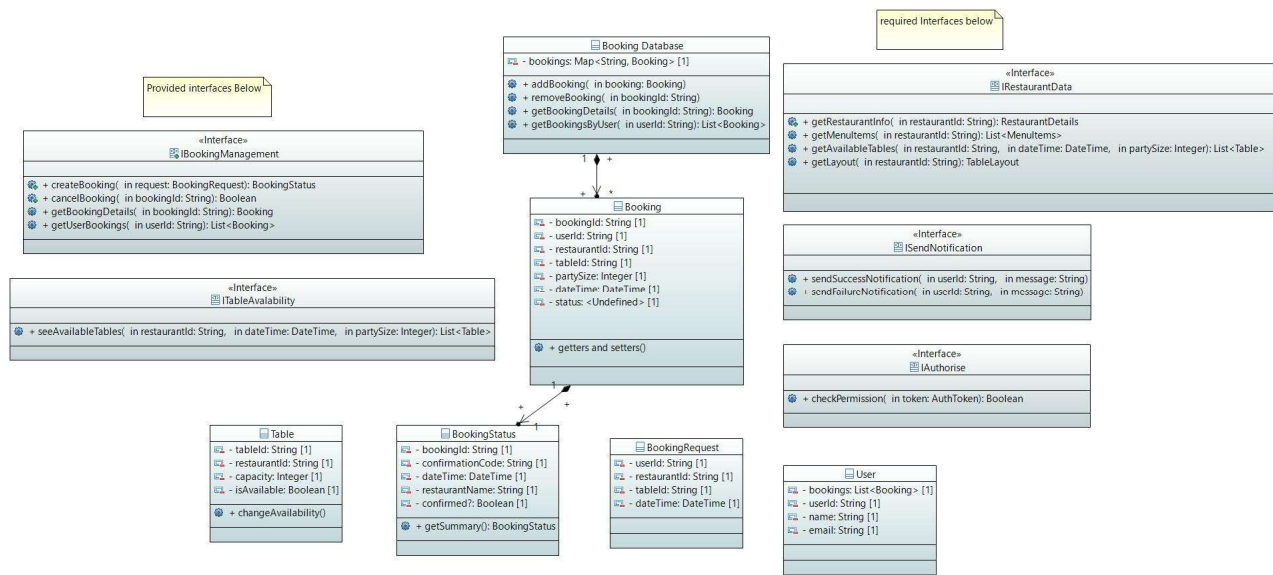
	Function	Update the specified order with new items. Returns the updated Order object if successful; otherwise, returns an error message. This operation modifies an existing order on the server.
Operation	Signature	viewOrderStatus(orderId: String): OrderStatus
	Function	Check the status of the specified order. Returns an OrderStatus object containing status information. This operation fetches and formats order status from the server.

Name	IFeedbackManagement	
Provider	Feedback Manager	
Operation	Signature	submitFeedback(restaurantId: String, rating: Int, comment: String): Feedback
	Function	Submit a review for the specified restaurant with the given rating and comment. Returns a Feedback object if submission is successful; otherwise, returns an error message. This client-side operation formats and sends feedback to the server.
Operation	Signature	viewPastFeedback(): List<Feedback>
	Function	Retrieve all feedback previously submitted by the current user. Returns a list of Feedback objects. This operation fetches and formats feedback data from the server.
Operation	Signature	uploadPhoto(feedbackId: String, photo: Photo): String
	Function	Upload a photo to be associated with the specified feedback. Returns a photo ID if successful; otherwise, returns an error message. This client-side operation handles image compression and upload to the server.

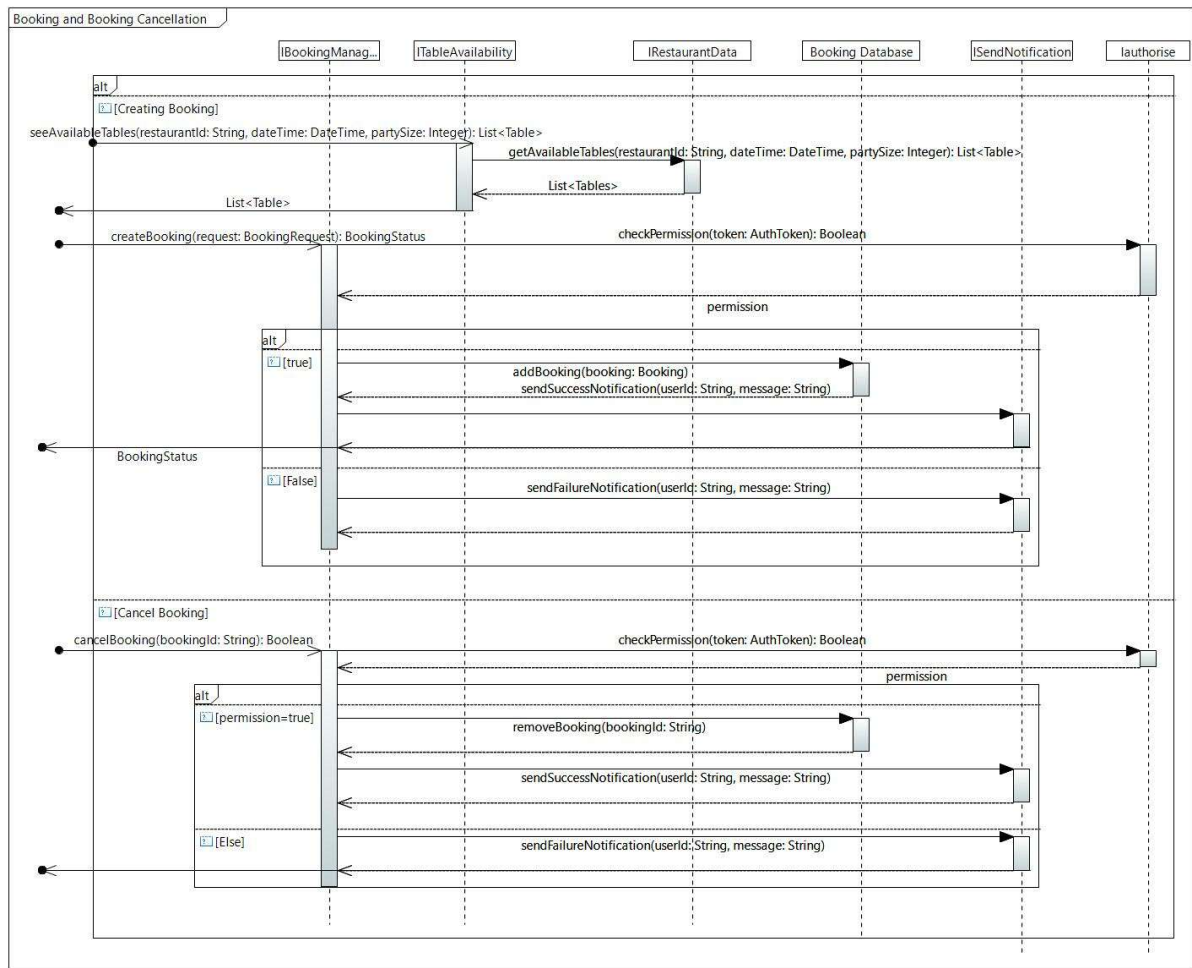
Name	IPaymentManagement	
Provider	Payment Manager	
Operation	Signature	payForOrder(orderId: String, method: PaymentMethod): PaymentResult
	Function	Process payment for the specified order using the given payment method. Returns a PaymentResult object indicating success or failure. This client-side operation securely handles payment information and communicates with the server.
Operation	Signature	viewPaymentHistory(): List<Payment>
	Function	Retrieve the payment transaction history for the current user. Returns a list of Payment objects. This operation fetches and formats payment records from the server.

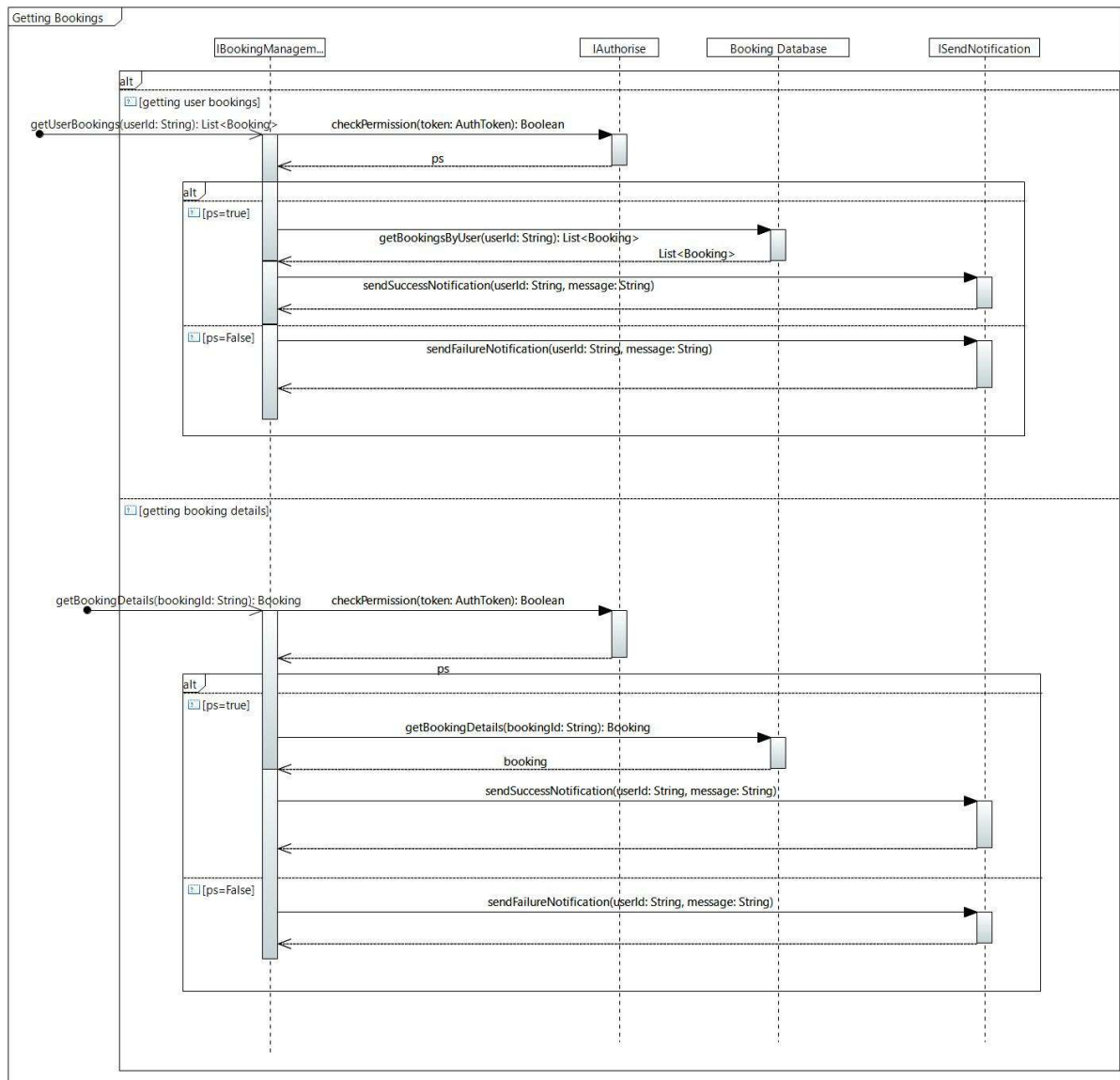
5. Detailed Design

5.1. Structural Model



5.2. Behaviour Model





The reason we have two pictures of sequence diagrams is simply combining them would have made it almost impossible to read.

6. Reflection

Having to redo the coursework one more time has given me the opportunity to increase my knowledge of software design and modelling. I look back at this experience with a positive attitude and find it a great learning opportunity and this time it was not as daunting of a task as it seemed the first time I tried the coursework.

Having said that, I believe for future students, recommending newer software for UML modelling would be much needed. I found myself struggling with especially Papyrus so many times and it took a great deal of effort to become more comfortable with it to tackle the tasks at hand. Although powerful, I think there are better software for the job. I know many of my course mates tried tools like draw.io or LucidChart but I understand they are not proper UML modelling platforms.

One struggle that I faced was having to travel between tasks to integrate the design as much as possible and make it coherent and it proved to be difficult and time consuming at times.